🐺 El Lobo Feroz (Programación Orientada a Objetos)

Objetivo conceptual

Pasar de múltiples clases “parecidas” a un **modelo unificado** mediante:

- estado (vivo)
- comportamiento común (cuantasCaloriasAporta)
- polimorfismo

🟢 PARTE 1 – Introducir estado de los objetos

“¿Qué pasa si el lobo intenta comerse dos veces a Caperucita?”

Ahí aparece el problema.

Solución conceptual

Todos los personajes deberían tener:

boolean vivo;

Implementación base (simple y clara)

Nueva clase: Personaje

```
package dominio;

public class Personaje {
   protected String nombre;
   protected boolean vivo;

   public Personaje(String nombre) {
      this.nombre = nombre;
```

```
        this.vivo = true;
    }

    public String getNombre() {
        return nombre;
    }

    public boolean estaVivo() {
        return vivo;
    }

    public void serComido() {
        this.vivo = false;
    }
}
```

🟡 PARTE 2 – Polimorfismo

Problema actual (el que ya tenés)

Hoy tu Lobo tiene:

```
comer(Chanchito)
comer(Caperucita)
comer(Oveja)
comer(Abuela)
```

👉 Esto escala mal

👉 Código duplicado

👉 No hay abstracción

Solución: método único

public void comer(Personaje personaje)

Pero... ¿cómo sabe cuántas calorías aporta?

Ahí entra el polimorfismo.

🔧 Diseño polimórfico

Clase abstracta

```
public abstract class Personaje {
   protected String nombre;
   protected boolean vivo;

   public Personaje(String nombre) {
      this.nombre = nombre;
      this.vivo = true;
   }

   public String getNombre() {
      return nombre;
   }

   public boolean estaVivo() {
      return vivo;
   }

   public void serComido() {
      this.vivo = false;
   }

   public abstract double cuantasCaloriasAporta();
}
```

Ejemplo: Caperucita

```
public class Caperucita extends Personaje {

   double peso;
   int manzanas;

   public Caperucita(String nombre, double peso, int manzanas) {
      super(nombre);
      this.peso = peso;
      this.manzanas = manzanas;
   }
```

```
    @Override
    public double cuantasCaloriasAporta() {
        return peso + 2 * manzanas;
    }
}
```

Ejemplo: Abuela

```
public class Abuela extends Personaje {

    public Abuela(String nombre) {
        super(nombre);
    }

    @Override
    public double cuantasCaloriasAporta() {
        return 0;
    }
}
```

🔴 Lobo con polimorfismo

```
public void comer(Personaje personaje) {

    if (!personaje.estaVivo()) {
        System.out.println(personaje.getNombre() + " ya fue comido.");
        return;
    }

    System.out.println("El lobo se comio a " + personaje.getNombre());

    this.calorias += personaje.cuantasCaloriasAporta();

    personaje.serComido();
}
```

🚀 Qué lográs con esto

Pasás de:

4 métodos distintos

a:

1 solo método (polimórfico)

🧠 Cómo explicarlo en clase (clave)

Podés decir esto:

“El lobo no necesita saber QUÉ es lo que está comiendo.
Solo necesita saber que es algo que:

- tiene nombre
- aporta calorías
- puede ser comido”

👉 Eso es **abstracción + polimorfismo**

🧪 Ejemplo en el main

```
Personaje p1 = new Caperucita("Caperucita", 65, 15);
Personaje p2 = new Abuela("Abuela");
Personaje p3 = new Oveja("Oveja", 30);

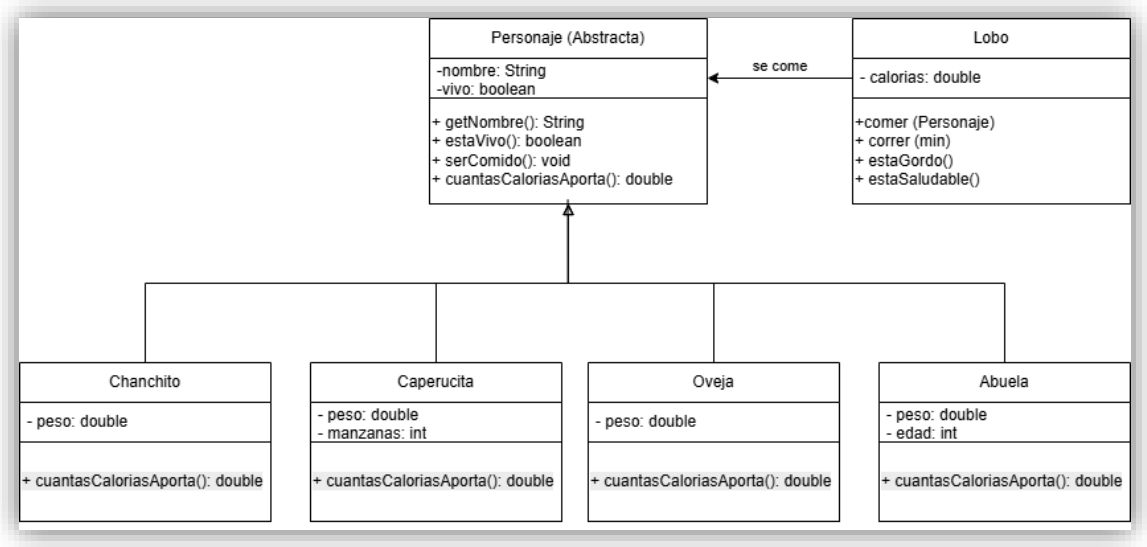
lobo.comer(p1);
lobo.comer(p2);
lobo.comer(p3);
```

👉 El mismo método
👉 Distinto comportamiento

💡 Preguntas potentes

- ¿Por qué el lobo no necesita saber si es una oveja o una abuela?
- ¿Qué pasa si agrego un nuevo personaje?
- ¿Tengo que modificar el lobo?

👉 Respuesta: NO → diseño abierto



🎯 Frase de cierre para la clase

"El polimorfismo no es magia.
Es diseñar objetos que responden al mismo mensaje de distintas formas."